

Chapter4 Notes

Unit IV: Cascading Style Sheets (CSS)

[8 Hrs.]

1. Introduction to CSS

- **CSS (Cascading Style Sheets)** — styles/formats HTML content (colors, layout, spacing, fonts).
 - Separates **content (HTML)** from **presentation (CSS)** — standard practice in every real project.
 - “Cascading” = rules applied in order of priority (specificity, source order, `!important`).
-

2. CSS Syntax

```
selector {  
  property: value;  
  property: value;  
}
```

Example:

```
p {  
  color: blue;  
  font-size: 16px;  
}
```

- **Selector** — targets HTML element(s).
 - **Property** — style aspect to change (color, font-size, margin...).
 - **Value** — setting for that property.
 - Rule block ends with `;` after each declaration.
-

3. Using CSS with HTML

Three ways to apply CSS — real projects almost always use **external**.

Inline CSS

```
<p style="color:red; font-size:14px;">Warning text</p>
```

Use case: quick one-off override, email templates (email clients often strip external CSS).

Internal CSS

```
<head>
  <style>
    p { color: green; }
  </style>
</head>
```

Use case: small single-page demo/prototype.

External CSS (most used in real projects)

```
<head>
  <link rel="stylesheet" href="styles.css">
</head>
```

```
/* styles.css */
p { color: navy; }
```

Why real projects use external: one file styles entire site, cached by browser (faster repeat visits), separates concerns, easy to maintain/scale (e.g., a company site with 50 pages uses one `styles.css`).

4. CSS Selectors

Selector	Syntax	Example	Real use
Element	<code>tag</code>	<code>p { }</code>	Style all paragraphs
Class	<code>.classname</code>	<code>.btn { }</code>	Reusable style — e.g., every button

Selector	Syntax	Example	Real use
ID	<code>#idname</code>	<code>#header { }</code>	Unique element — e.g., page header
Universal	<code>*</code>	<code>* { margin:0; }</code>	CSS reset
Descendant	<code>A B</code>	<code>nav a { }</code>	Links inside nav only
Child	<code>A > B</code>	<code>ul > li { }</code>	Direct children only
Pseudo-class	<code>:state</code>	<code>a:hover { }</code>	Style on interaction
Pseudo-element	<code>::part</code>	<code>p::first-line { }</code>	Style part of element
Attribute	<code>[attr=val]</code>	<code>input[type="email"] { }</code>	Style specific input types
Group	<code>A, B</code>	<code>h1, h2 { }</code>	Apply same style to multiple

Real project example — navbar link states:

```

nav a {
  color: white;
  text-decoration: none;
}
nav a:hover {
  color: #ffcc00;      /* highlight on mouse hover */
}
nav a:active {
  color: red;
}
.btn-primary {
  background-color: #2563eb;
  color: white;
}
#site-logo {
  font-size: 24px;
  font-weight: bold;
}
input[type="email"] {
  border: 1px solid #ccc;
}

```

5. CSS Comments

```
/* This is a CSS comment - not rendered on page */  
p {  
  color: blue; /* text color */  
}
```

Used in real projects to mark sections: `/* ===== Header Styles ===== */`.

6. CSS Properties

6.1 Backgrounds

```
.hero-banner {  
  background-color: #1e293b;  
  background-image: url("banner.jpg");  
  background-size: cover;  
  background-position: center;  
  background-repeat: no-repeat;  
}
```

Real use: landing page hero sections, card backgrounds.

6.2 Border, Margin

```
.card {  
  border: 1px solid #ddd;  
  border-radius: 8px; /* rounded corners - used everywhere: buttons, cards, modals */  
  margin: 20px auto; /* auto centers block horizontally */  
}
```

6.3 Padding

```
.btn {  
  padding: 10px 20px; /* top/bottom 10px, left/right 20px */  
}
```

Real use: buttons need padding so text doesn't touch edges — every button on every site uses this.

6.4 Height / 6.5 Width

```
.product-image {
  width: 100%;
  max-width: 300px;
  height: auto; /* keeps aspect ratio - prevents image distortion */
}
```

6.6 Color (Color Wheel)

```
.text-danger { color: red; }
.text-danger { color: #ff0000; }
.text-danger { color: rgb(255, 0, 0); }
.text-danger { color: rgba(255, 0, 0, 0.7); } /* with transparency - used for overlays */
.text-danger { color: hsl(0, 100%, 50%); }
```

Format	Example	Real use
Named	<code>red</code> , <code>navy</code>	Quick prototyping
Hex	<code>#ff0000</code>	Most common in real design systems (brand colors)
RGB/RGBA	<code>rgb(255,0,0)</code> / <code>rgba(...,0.5)</code>	RGBA used for overlays, transparent backgrounds
HSL	<code>hsl(0,100%,50%)</code>	Easy to adjust lightness/saturation programmatically (theme generation)

Color wheel basics: primary (red, blue, yellow) → secondary (mix of two primary) → complementary colors (opposite on wheel, used for accents/CTAs) → used by designers to pick harmonious brand palettes (e.g., blue primary + orange CTA button = complementary contrast, common pattern in real UI).

7. Text

```
p {
  color: #333;
  text-align: justify;
  text-decoration: none;
  text-transform: uppercase;
  letter-spacing: 1px;
}
```

```
text-shadow: 1px 1px 2px gray;
}
```

Real use: `text-decoration: none` universally applied to remove underline from `<a>` links in navbars.

8. Font

```
body {
  font-family: "Segoe UI", Arial, sans-serif; /* fallback fonts if first
    unavailable */
  font-size: 16px;
  font-weight: 400;
  font-style: normal;
}
h1 {
  font-family: 'Poppins', sans-serif;
  font-weight: 700;
}
```

Real project: Google Fonts imported via `<link>` in `<head>`, then applied with `font-family` — nearly every modern website (blogs, e-commerce, SaaS landing pages) does this.

```
<link href="https://fonts.googleapis.com/css2?family=Poppins&display=swap"
      rel="stylesheet">
```

9. Alignment

```
.container {
  text-align: center; /* aligns inline/text content */
}
.flex-container {
  display: flex;
  justify-content: center; /* horizontal alignment in flexbox */
  align-items: center; /* vertical alignment in flexbox */
}
```

Real use: centering a login form box in middle of screen — extremely common pattern:

```
.login-wrapper {
  display: flex;
  justify-content: center;
```

```
align-items: center;
height: 100vh;
}
```

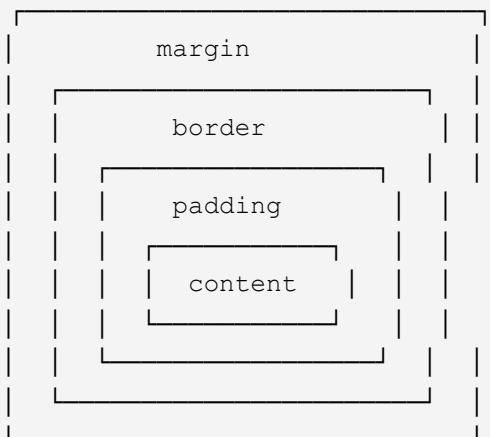
10. Line Height

```
p {
  line-height: 1.6;  /* improves readability of paragraphs */
}
```

Real use: blog/article body text almost always sets `line-height: 1.5-1.8` for comfortable reading.

11. Box Model

Every HTML element is a box made of 4 layers (inside-out):



```
.box {
  width: 300px;
  padding: 20px;
  border: 2px solid black;
  margin: 10px;
  box-sizing: border-box;  /* width includes padding+border - prevents layout
                           breaking */
}
```

`box-sizing: border-box` — used almost universally in real projects (often set globally):

```
* { box-sizing: border-box; }
```

Without it, `width` only refers to content, so padding/border add extra size and break layouts — a very common bug in beginner projects.

12. Working with Images

```
img {
  max-width: 100%;
  height: auto;           /* responsive image - never overflows container */
  border-radius: 8px;
  object-fit: cover;      /* crops image to fill box without distortion - used in
                           profile pics/thumbnails */
}

.avatar {
  width: 50px;
  height: 50px;
  border-radius: 50%;     /* circular profile picture - used on every social
                           media/dashboard */
  object-fit: cover;
}
```

13. Layout and Positioning

Position property

Value	Behavior	Real use
<code>static</code>	Default, normal flow	Regular content
<code>relative</code>	Offset from normal position, keeps space	Small nudges, positioning context for children
<code>absolute</code>	Removed from flow, positioned relative to nearest positioned ancestor	Dropdown menus, tooltips, badges on icons
<code>fixed</code>	Positioned relative to viewport, stays on scroll	Sticky navbar, “back to top” button, chat widget
<code>sticky</code>	Toggles between relative/fixed based on scroll	Sticky table headers, sticky sidebar

```
/* Real project: notification badge on cart icon */
.cart-icon { position: relative; }
```

```

.badge {
  position: absolute;
  top: -5px;
  right: -5px;
  background: red;
  color: white;
  border-radius: 50%;
  padding: 2px 6px;
}

/* Real project: fixed navbar that stays visible while scrolling */
.navbar {
  position: fixed;
  top: 0;
  width: 100%;
  background: white;
  z-index: 1000;
}

```

Float and Clear (older technique, still asked in exams)

```

.image-left {
  float: left;
  margin-right: 15px;
}

.clearfix::after {
  content: "";
  display: block;
  clear: both;
}

```

Real use: wrapping text around an image (blog article style) — mostly replaced today by Flexbox/Grid but still common in legacy code and exams.

Display property

```

.item { display: block; }      /* takes full width, new line - div, p, h1 */
.item { display: inline; }    /* no new line, no width/height - span, a */
.item { display: inline-block; } /* inline but can set width/height */
.item { display: none; }      /* hides element completely - used for mobile menu
                               toggle, modals */

.container { display: flex; }  /* flexible box layout - navbars, cards row */
.container { display: grid; } /* grid layout - dashboards, photo galleries */

```

Real project example — Flexbox navbar (used everywhere):

```
.navbar {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  padding: 15px 30px;  
}
```

Real project example — Grid product gallery (e-commerce):

```
.product-grid {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  gap: 20px;  
}
```

14. Media Query

- Applies different CSS rules based on screen width/device — core of responsive design.

```
/* Default: desktop */  
.container {  
  width: 1200px;  
}  
  
/* Tablet */  
@media (max-width: 768px) {  
  .container { width: 100%; padding: 0 15px; }  
  .product-grid { grid-template-columns: repeat(2, 1fr); }  
}  
  
/* Mobile */  
@media (max-width: 480px) {  
  .navbar { flex-direction: column; }  
  .product-grid { grid-template-columns: 1fr; }  
}
```

Real project use: every commercial website (Amazon, Daraz, any company site) uses this exact breakpoint pattern (desktop → tablet ~768px → mobile ~480px) to reflow layout.

15. CSS Website Layout

Real full-page layout combining box model + flexbox/grid + media queries:

```
<!DOCTYPE html>
<html>
<head>
<style>
  * { box-sizing: border-box; margin: 0; padding: 0; }
  body { font-family: Arial, sans-serif; line-height: 1.6; }

  header {
    background: #1e293b;
    color: white;
    padding: 15px 30px;
    display: flex;
    justify-content: space-between;
    align-items: center;
    position: fixed;
    top: 0; width: 100%;
  }
  header nav a {
    color: white;
    text-decoration: none;
    margin-left: 20px;
  }

  .hero {
    margin-top: 70px;
    background: url("banner.jpg") center/cover no-repeat;
    height: 400px;
    display: flex;
    justify-content: center;
    align-items: center;
    color: white;
    text-align: center;
  }

  .products {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 20px;
    padding: 30px;
  }
  .product-card {
    border: 1px solid #ddd;
    border-radius: 8px;
    padding: 15px;
    text-align: center;
  }

  footer {
    background: #1e293b;
```

```

        color: white;
        text-align: center;
        padding: 20px;
    }

    @media (max-width: 768px) {
        .products { grid-template-columns: repeat(2, 1fr); }
    }

    @media (max-width: 480px) {
        .products { grid-template-columns: 1fr; }
        header { flex-direction: column; }
    }
</style>
</head>
<body>
    <header>
        <h1>ShopEasy</h1>
        <nav>
            <a href="#">Home</a>
            <a href="#">Shop</a>
            <a href="#">Contact</a>
        </nav>
    </header>

    <section class="hero">
        <h2>Big Sale - Up to 50% Off</h2>
    </section>

    <section class="products">
        <div class="product-card"><h3>Shoes</h3><p>Rs. 3,000</p></div>
        <div class="product-card"><h3>Watch</h3><p>Rs. 5,000</p></div>
        <div class="product-card"><h3>Bag</h3><p>Rs. 2,000</p></div>
    </section>

    <footer>
        <p>&copy; 2026 ShopEasy. All rights reserved.</p>
    </footer>
</body>
</html>

```

This single example demonstrates: box model, flexbox, grid, positioning, media queries, backgrounds, typography — essentially a mini real e-commerce homepage.

Quick Recap (Exam Points)

- CSS = styling language; syntax = `selector { property: value; }`.
- 3 ways to apply: inline, internal, external (external preferred in real projects).

- Selectors: element, class (`.`), id (`#`), pseudo-class (`:hover`), attribute (`[type=]`).
 - Box model (inside-out): content → padding → border → margin; `box-sizing: border-box` prevents size bugs.
 - Position: static, relative, absolute, fixed, sticky.
 - Display: block, inline, inline-block, none, flex, grid.
 - Flexbox = 1D layout (navbars); Grid = 2D layout (galleries/dashboards).
 - Media queries = responsive breakpoints (`@media (max-width: ...)`).
 - Colors: named, hex, rgb/rgba, hsl.
-

Possible Exam Questions

Short Answer / Conceptual

1. What is CSS? Explain the three ways of applying CSS to HTML with examples.
2. Differentiate between class selector and ID selector.
3. Explain the CSS Box Model with a diagram.
4. What is the difference between `margin` and `padding` ?
5. Explain the difference between `position: relative` and `position: absolute` .
6. What is `box-sizing: border-box` and why is it used?
7. Differentiate between `display: block` , `inline` , and `inline-block` .
8. What is a media query? Why is it important in responsive web design?
9. Explain any four color formats used in CSS with examples.
10. What is the difference between Flexbox and Grid layout?
11. What is `z-index` ? When is it used?
12. Differentiate `float` and `position` for layout purposes.

Long Answer / Programming

1. Design a responsive navbar using Flexbox that stacks vertically on mobile screens (using media query).
2. Write CSS to create a product card with image, title, price, border-radius, and box-shadow.
3. Explain and demonstrate the CSS Box Model using a code example with `width` , `padding` , `border` , and `margin` .
4. Create a 3-column responsive grid layout that becomes 1-column on mobile using Grid and media queries.
5. Write CSS code to design a fixed header that remains visible while scrolling, along with a “back to top” button using `position: fixed` .
6. Design a complete simple webpage (header, hero section, content grid, footer) using external CSS with responsive breakpoints.

7. Explain with example how pseudo-classes (`:hover` , `:active` , `:focus`) enhance user interaction on a website.

True/False or Fill in the Blanks

1. External CSS is preferred over inline CSS for large real-world websites. (True)
2. `position: fixed` element moves along with page scroll. (False)
3. `display: none` hides an element but still reserves its space in layout. (False)
4. _____ property is used to create a responsive grid layout in CSS. (`display: grid`)
5. The correct order of the CSS box model from inside to outside is: content, _____, border, margin. (padding)
6. _____ unit in media queries is commonly used to define breakpoint width. (px)